

Recurrent Neural Network

Content

- Introduction to Sequential Data
 - What is sequential data (text, speech, time-series, video frames)
 - Why traditional feedforward networks fail on sequence dependencies
 - Concept of memory in model
- Basic RNN Structure
 - Hidden state, weights, recurrence relation
 - Forward pass equations
 - Backpropagation through time (BPTT) (intuition)
- Gradient Issues
- Why gradients vanish or explode
- Time Series Forecasting with RNNs
 - Time series characteristics
 - Stationarity and trends
 - Creating sliding windows for supervised learning
 - Train-test split strategies for temporal data

Introduction to Sequential Data

Many real-world datasets are not simply made up of isolated, independent observations. Instead, they follow a specific order in which each element's position carries important information. This ordered arrangement defines what we call *sequential data*. In mathematical terms, a sequence can be represented as an ordered set

$$x=(x_1,x_2,\dots,x_T)$$

where x_t denotes the value at the t _th time step (or position) and T represents the total number of elements in the sequence.

A key property of such data is that the sequence order is essential. Changing the arrangement of elements alters the interpretation and structure of the data. In other words, permuting the elements leads to a completely different meaning compared to the original sequence.



Speech



Text



Music



Protein & DNA Sequence



Monthly Demand data

Some examples of Sequential Data

Text Data

Human language is one of the clearest examples of sequential data. Take the sentence, *“The quick brown fox jumps over the lazy dog.”* Its meaning comes from the particular order in which the words appear. If we rearrange it as *“Lazy jumps dog the over brown quick fox the,”* the sentence loses its sense and becomes meaningless. This shows that the position of each word in relation to others plays a crucial role in conveying meaning.

Common applications of sequential text data include:

Sentiment analysis: determining whether a review expresses positive or negative opinion

Machine translation: converting text from one language to another (e.g., English to French)

Text generation: producing new content such as articles or computer code

Time Series Data

Data points recorded over time form a sequence. Examples include:

Daily stock prices

Hourly temperature readings

Monthly sales figures

Electrocardiogram (ECG) signals

In these cases, the order is important. The value of a stock price today depends on its previous values. Weather patterns emerge from the sequence of temperature, pressure, and humidity readings over time. Predicting future values (forecasting) relies heavily on understanding these past temporal dependencies.

Audio Data



Sound naturally follows a sequential structure. An audio signal can be described as a series of amplitude measurements taken at fixed time intervals (for example, 44,100 samples per second in CD-quality recordings).

The sequence in which these values appear determines important characteristics such as pitch, tempo, and overall sound patterns. This is what allows us to distinguish between speech, music, and background noises.

If these samples are randomly reordered, the original sound pattern is lost and the audio becomes unrecognizable.

Video Data

Video can be seen as a sequence of images (frames). While each frame is a spatial grid of pixels, the sequence of frames over time creates motion and narrative.

Tasks like action recognition in videos depend on analyzing how objects or scenes change across consecutive frames.

Biological Sequences



In the field of bioinformatics, genetic and protein data are naturally represented as sequences. A DNA molecule consists of an ordered chain of nucleotides (A, C, G, and T), while proteins are formed from sequences of amino acids.

The arrangement of these elements is crucial because it governs the molecule's structure, behavior, and biological role. Tasks such as gene identification and protein structure prediction depend on studying and interpreting these ordered patterns.

Characteristics of Sequential Data

Importance of Sequence Structure

A key property of sequential data is that the position of each element plays a vital role in defining its interpretation. Unlike a group of independent attributes used to describe an object—such as a house, where listing features like size, number of rooms, or location in any order does not alter the object itself—the arrangement of values in a sequence often determines the underlying pattern or message.

Consider the following two statements:

“The cat chased the mouse.”

“The mouse chased the cat.”

Both sentences contain the same words, yet their different word orders communicate entirely different events.

Temporal Relationships in Sequences



Another important aspect of sequential data is the presence of temporal relationships between its elements. This means that a value at a given position in a sequence is often influenced by values that appeared earlier (and in some cases, later). These relationships can span different lengths depending on the nature of the data.

Short-term relationships:

In natural language, nearby words often influence each other directly. For example, in the phrase *“the cat sits,”* the form of the verb is closely tied to the noun that comes just before it. Here, the dependency exists over a very small distance in the sequence.

Long-term relationships:

Now consider the sentence:

“The report on machine learning trends, which was published last month after extensive research across multiple industries, highlights the growing adoption of sequence models.”

In this case, the verb *“highlights”* is connected to the subject *“report,”* even though many words separate them.

Learning such distant relationships is difficult because the relevant information needed to interpret or predict a value at time step t may have occurred far earlier in the sequence. Handling these extended dependencies is one of the major challenges in modeling sequential data.

Variable Length

Unlike structured datasets where each sample typically has a fixed number of features (e.g., columns in a table), sequences often have **variable lengths**. Sentences can be short or long, time series measurements might cover different durations, and audio clips vary in length.

This poses practical challenges for machine learning models, especially when processing data in batches. Standard deep learning frameworks often expect input tensors with uniform shapes within a batch.

Why Feedforward Networks Are Inadequate for Sequential Data

Conventional feedforward neural networks are designed to process each input independently, without considering any relationship to previous or future inputs. They assume that all samples are unrelated and that the order in which data is presented does not matter.

This assumption works well for tasks such as image classification or tabular prediction, but it breaks down when dealing with sequences.

In sequential problems, the output at a given step often depends on earlier values in the sequence. For example, predicting tomorrow's temperature requires knowledge of temperature trends from previous days.

A feedforward network, however, treats each input in isolation and has no built-in mechanism to retain information from past inputs. As a result, it cannot naturally capture temporal patterns or dependencies that span across time steps.

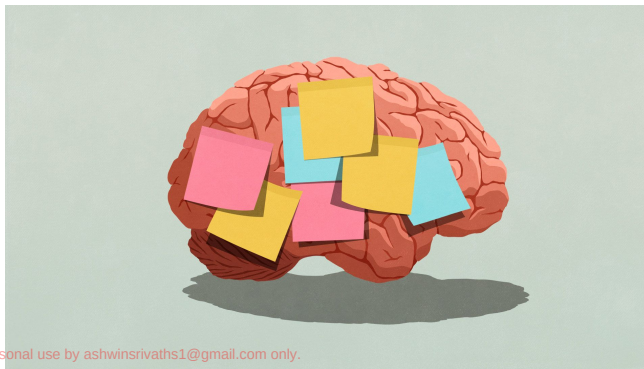
Concept of Memory in a Model

To effectively model sequential data, a learning system must be able to store and use information from earlier parts of the sequence. This requirement introduces the idea of **memory** within the model.

Memory allows the network to retain relevant details from past inputs and use them when processing new ones.

This is typically achieved by maintaining an internal state that is updated as new inputs arrive. At each step, the model combines the current input with information stored from previous steps, enabling it to build a contextual understanding of the sequence over time.

In this way, memory provides continuity across time steps and allows the model to learn patterns that extend beyond a single observation.



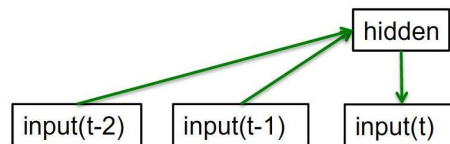
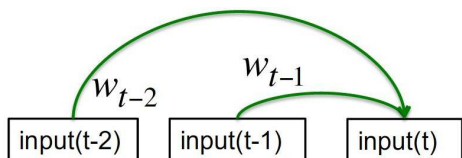
Ways to Deal with Sequence Labeling

1. Autoregressive models

Predict the next term in a sequence from a fixed number of previous terms using delay taps.

1. Feed-forward neural nets

These generalize autoregressive models by using one or more layers of non-linear hidden units



Memoryless models: limited word-memory window; hidden state cannot be used efficiently.

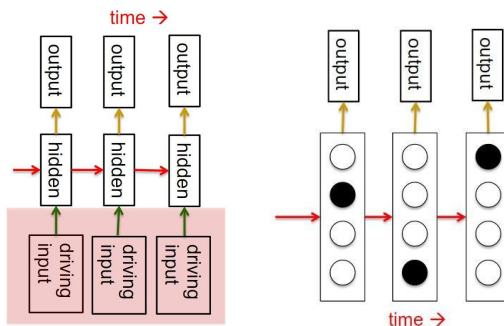
Ways to Deal with Sequence Labeling

1. Linear Dynamical Systems

These are generative models. They have a real-valued hidden state that cannot be observed directly.

1. Hidden Markov Models

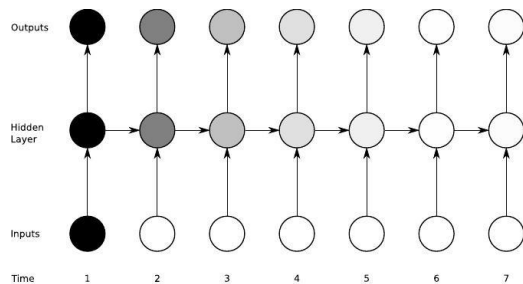
Have a discrete one-of-N hidden state. Transitions between states are stochastic and controlled by a transition matrix. The outputs produced by a state are stochastic.



Memoryful models,
time-cost to infer the hidden state distribution.

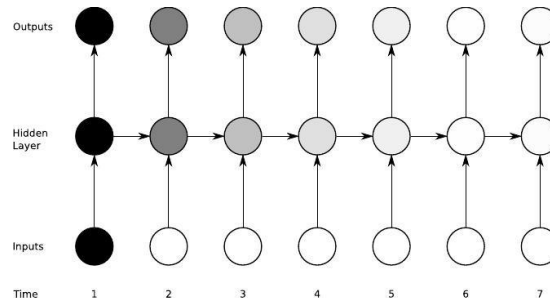
1. Finally, the RNN model!

1. Update the hidden state in a deterministic nonlinear way.
2. In the simple speaking case, we send the chosen word back to the network as input.



Ways to Deal with Sequence Labeling

1. RNNs are very powerful, because they:
 1. Distributed hidden state that allows them to store a lot of information about the past efficiently.
 2. Non-linear dynamics that allows them to update their hidden state in complicated ways.
 3. No need to infer hidden state, pure deterministic.
 4. Weight sharing



Recurrent Neural Network (RNN)

Basic definition:

A neural network with feedback connections.

X: Input

O: Output

S: Hidden state

Weights: $[U, V, W]$

Learned during training

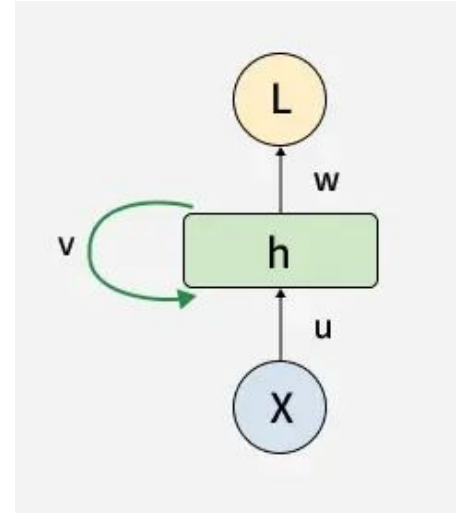
Core Elements of Recurrent Neural Networks

Recurrent Neural Networks are built around two primary components, which we will focus on in this discussion.

1. Recurrent Units

The basic building block of an RNN is the **recurrent unit**. Unlike neurons in a feedforward network, these units maintain an internal state that stores information from earlier time steps in a sequence. This internal state acts as a form of memory, enabling the network to retain context from previous inputs.

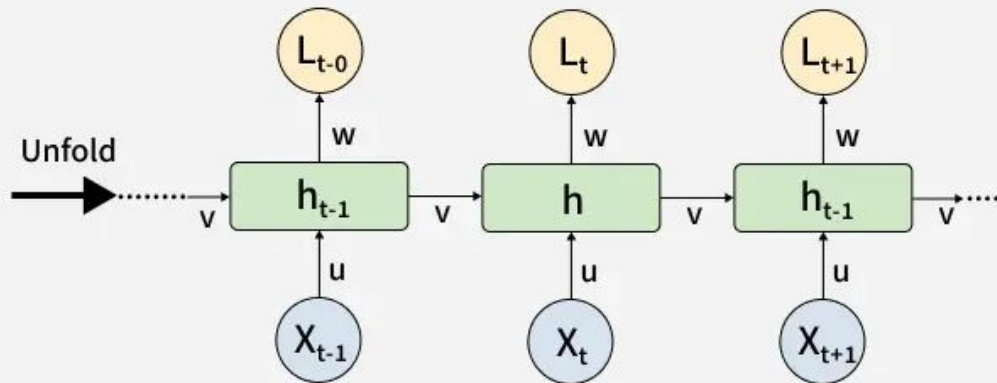
By passing this hidden state forward at each step, recurrent units allow the model to incorporate past information when processing new inputs. This feedback mechanism helps the network learn relationships that evolve over time and makes it suitable for handling sequential patterns.



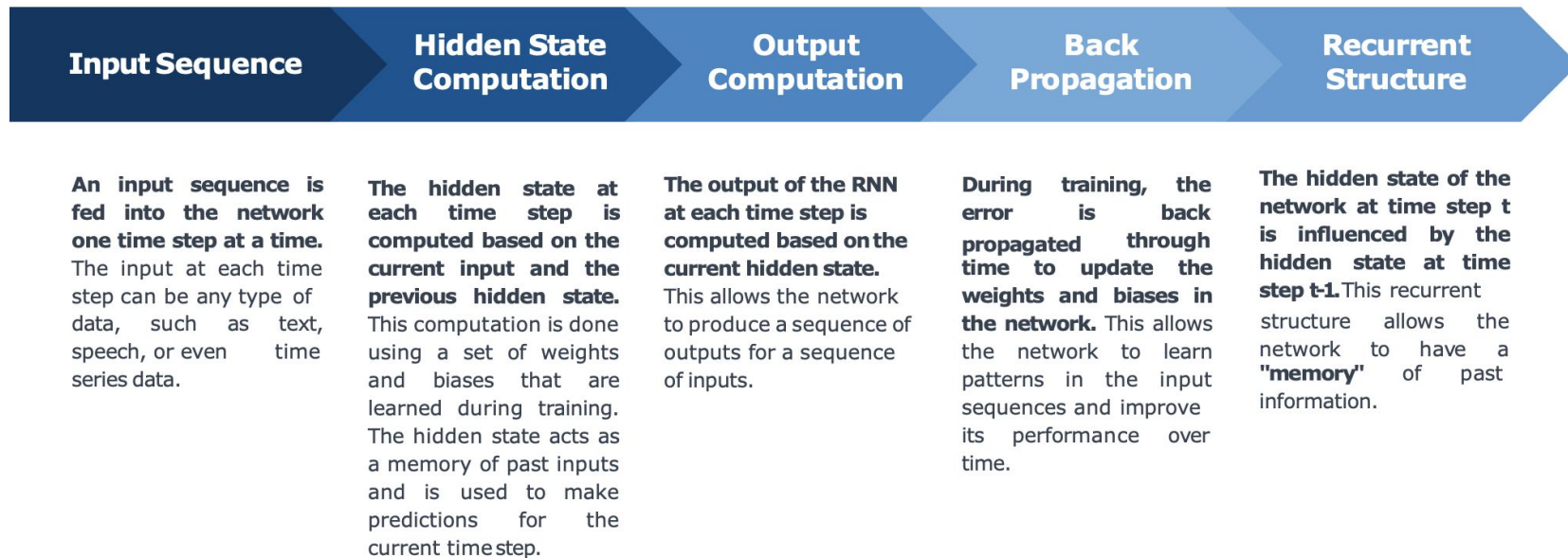
RNN Unfolding

RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.

This unrolling enables backpropagation through time (BPTT) a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn dependencies within sequential data.



How do RNNs work?



Basic RNN Structure

A Recurrent Neural Network (RNN) processes data **one time step at a time** and keeps track of past information using a hidden state.

Concept:

Output at time t depends on:

current input

past hidden state

Basic RNN Architecture Components

Notation:

- Input at time t : $x_t \in \mathbb{R}^{d_x}$
- Hidden state at time t : $h_t \in \mathbb{R}^{d_h}$
- Output at time t : $y_t \in \mathbb{R}^{d_y}$
- Weight matrices:
 - Input-to-hidden: $W_{xh} \in \mathbb{R}^{d_h \times d_x}$
 - Hidden-to-hidden (recurrent): $W_{hh} \in \mathbb{R}^{d_h \times d_h}$
 - Hidden-to-output: $W_{hy} \in \mathbb{R}^{d_y \times d_h}$
- Bias vectors:
 - Hidden bias: $b_h \in \mathbb{R}^{d_h}$
 - Output bias: $b_y \in \mathbb{R}^{d_y}$



Comparison with Feedforward Network

Feedforward Network:

$$y = f(x)$$

RNN:

$$y_t = f(x_t, h_{t-1})$$

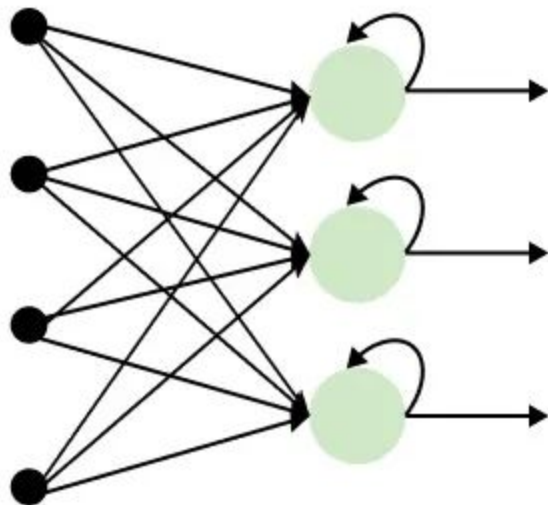
Difference:

Feedforward NN → no memory

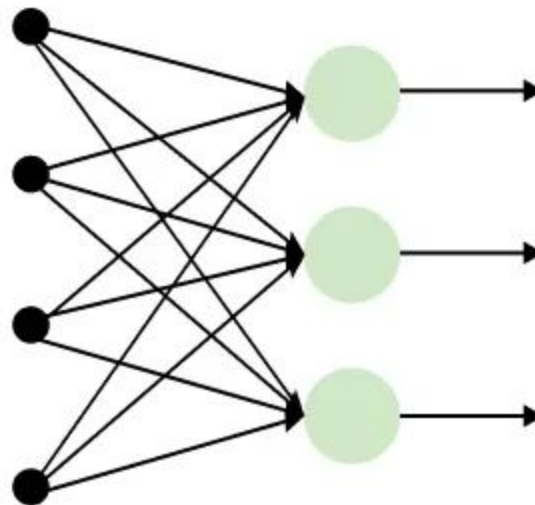
RNN → remembers past using hidden state

RNN introduces a **feedback connection** so past information can influence present predictions.

(a) Recurrent Neural Network



(b) Feed-Forward Neural Network



Hidden State (Memory)



Title: Hidden State h_t

Definition:

The hidden state is an internal representation that stores information from previous time steps.

Role of Hidden State:

- Acts as short-term memory

- Encodes past sequence information

- Updated at every time step

h_t = memory at time t

Intuition:

Hidden state summarizes “what the network has seen so far.”

Recurrence Relation (Core Equation)

Hidden State Update:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Output Equation:

$$y_t = W_{hy}h_t + b_y$$

Explanation:

x_t = current input

h_{t-1} = previous memory

$\tanh$ = activation function

h_t = updated memory

Why It Is Called “Recurrent”

$$h_t \rightarrow h_{t+1} \rightarrow h_{t+2}$$

The same weights are reused at each step.
 W_x, W_h, W_y (shared across time)

Key Benefit:

Parameter sharing

Works for variable-length sequences

Unrolling the RNN in Time

Unrolled view:

$$h_1 = \tanh(W_{xh}x_1 + W_{hh}h_0 + b_h)$$

$$h_2 = \tanh(W_{xh}x_2 + W_{hh}h_1 + b_h)$$

$$h_3 = \tanh(W_{xh}x_3 + W_{hh}h_2 + b_h)$$

⋮



$$h_T = \tanh(W_{xh}x_T + W_{hh}h_{T-1} + b_h)$$

Interpretation:

RNN can be seen as a deep network across time steps.

Information Flow in RNN

At time step t :

1. Take input x_t
2. Combine with previous hidden state h_{t-1}
3. Compute new hidden state h_t
4. Produce output y_t

Pipeline:

$$(x_t, h_{t-1}) \rightarrow h_t \rightarrow y_t$$

Example (Time Series)

Suppose:

$$x = [10, 12, 15, 18]$$

Prediction task:

$$[10, 12, 15] \rightarrow 18$$

RNN processes:

- step 1: $x_1 = 10 \rightarrow h_1$
- step 2: $x_2 = 12 \rightarrow h_2$
- step 3: $x_3 = 15 \rightarrow h_3$
- use h_3 to predict next value

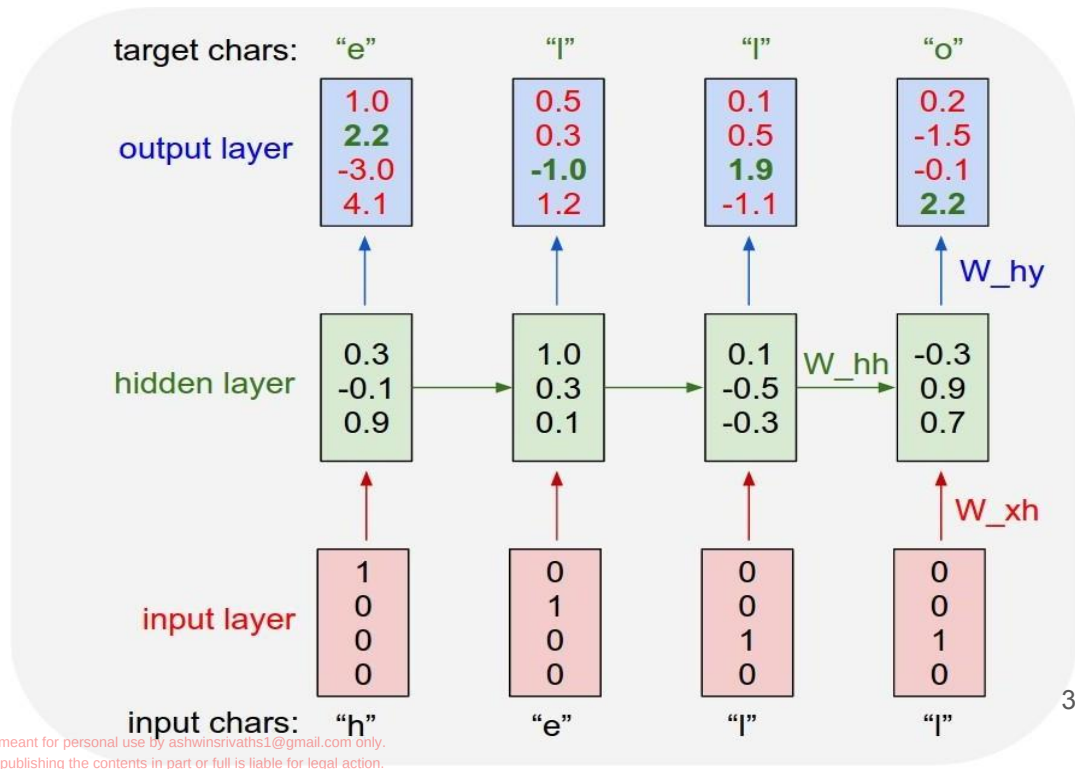
RNN - Example

Character level language model:

- Given previous and current characters, predict the character in the next sequence.

Let

- Vocabulary: [h,e,l,o]
- One-hot representations
 - $h = [1\ 0\ 0\ 0]$
 - $e = [0\ 1\ 0\ 0]$
 - $l = [0\ 0\ 1\ 0]$
 - $o = [0\ 0\ 0\ 1]$



Training of RNNs

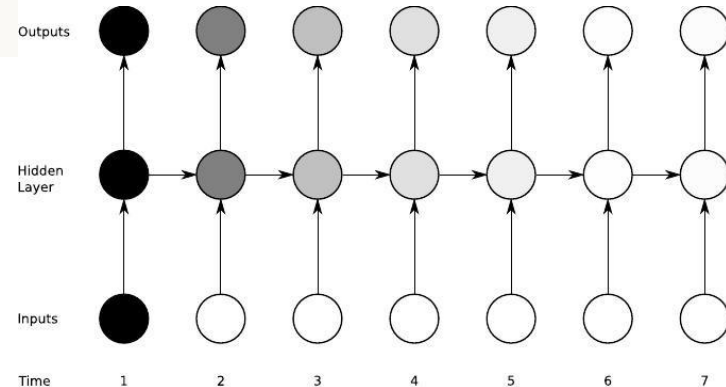
Vanilla Forward Pass

1. The complete sequence of hidden activations can be calculated by starting at $t = 1$ and recursively applying the three equations, incrementing t at each step.

$$a_h^t = \sum_{i=1}^{d_x} W_{xh}^{(i)} x_t^{(i)} + \sum_{j=1}^{d_h} W_{hh}^{(j)} h_{t-1}^{(j)} + b_h$$

$$a_h^t = W_{xh} x_t + W_{hh} h_{t-1} + b_h$$

$$h_t = \tanh(a_h^t)$$



Vanilla Backward Pass

1. Given the partial derivatives of the objective function with respect to the network outputs, we now need the derivatives with respect to the weights.
2. We focus on BPTT since it is both conceptually simpler and more efficient in computation time (though not in memory). Like standard back-propagation, BPTT consists of a repeated application of the chain rule.

$$a_h^t = \sum_{i=1}^{d_x} W_{xh,ji} x_t^{(i)} + \sum_{j'=1}^{d_h} W_{hh,jj'} h_{t-1}^{(j')} + b_h^{(j)}$$

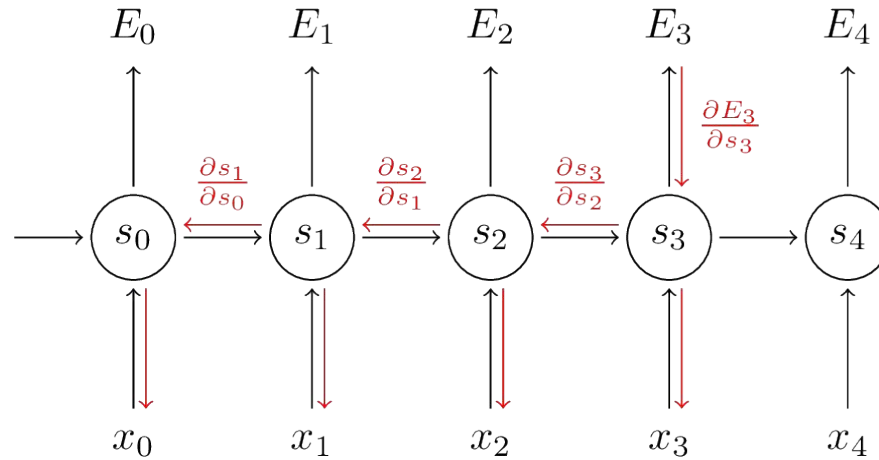
$$a_h^t = W_{xh}x_t + W_{hh}h_{t-1} + b_h$$

$$h_t = \tanh(a_h^t)$$

1. Vanilla Forward Pass

1. The forward pass of a vanilla RNN
 1. The same as that of an MLP with a single hidden layer
 2. Except that activations arrive at the hidden layer from both the current external input and the hidden layer activations one step back in time.

Backpropagation Through Time (BPTT).

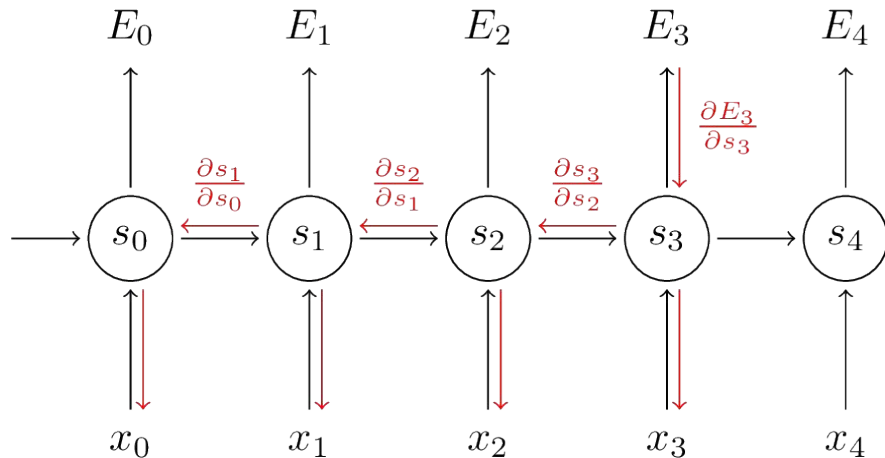


BackPropagation Through Time (BPTT)

Error for an instance = Sum of errors at each time step of the instance

Gradient of error

$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}$$



BackPropagation Through Time (BPTT)

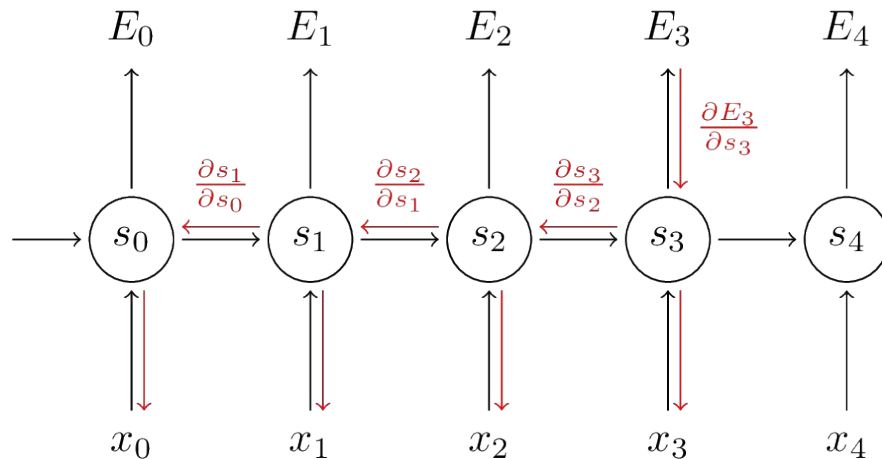
For V

$$\frac{\partial E_3}{\partial V} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial V}$$

For W (Similarly for U)

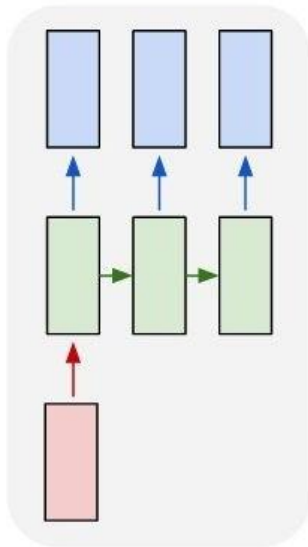
$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial W}$$

$$\frac{\partial E_3}{\partial W} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial W}$$



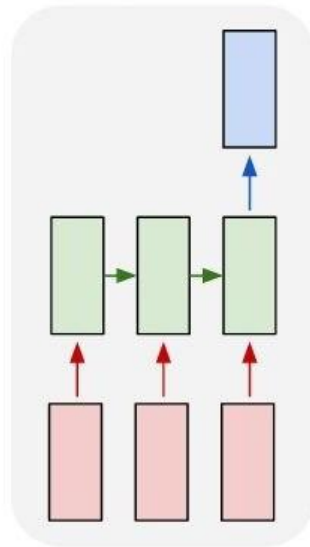
Different configurations of RNNs

one to many



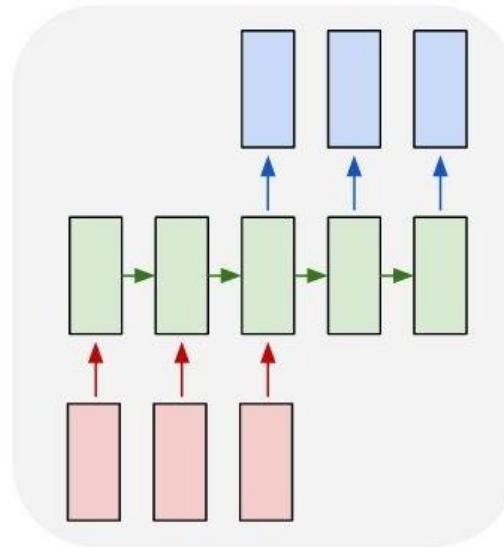
**Image
Captioning**

many to one



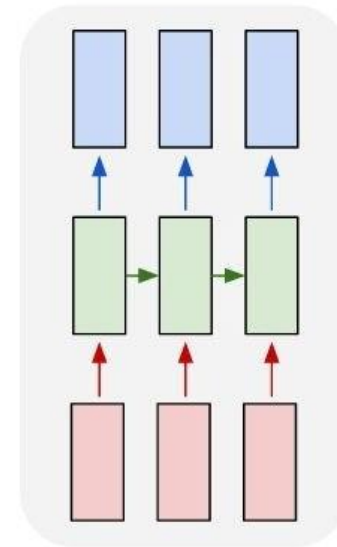
**Sentiment
Analysis**

many to many



**Machine
Translation**

many to many

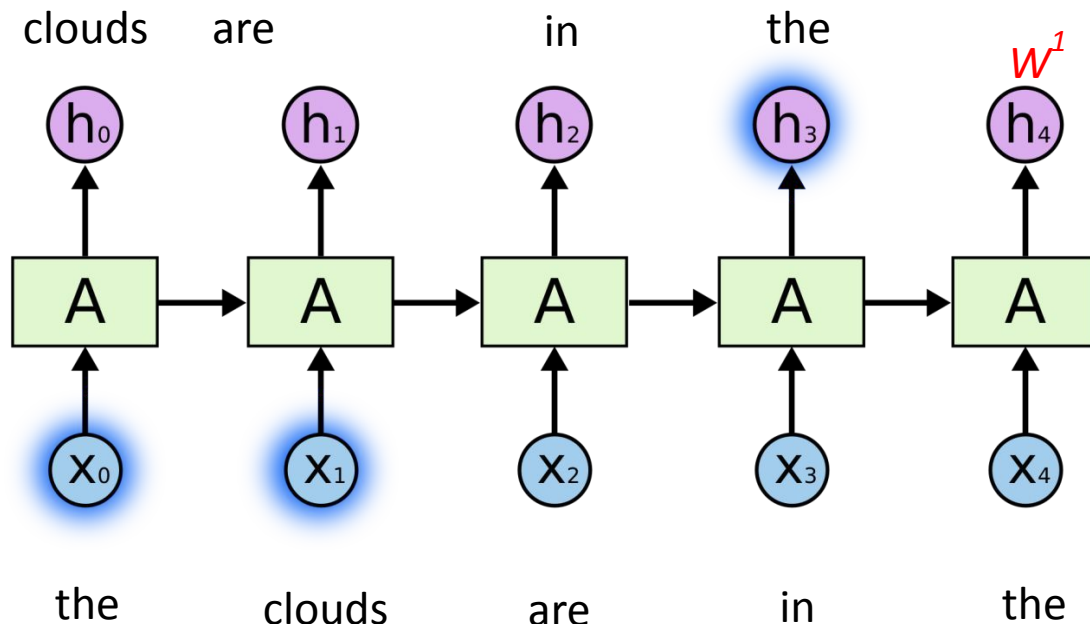


**Language
modelling**

Problems with RNNs

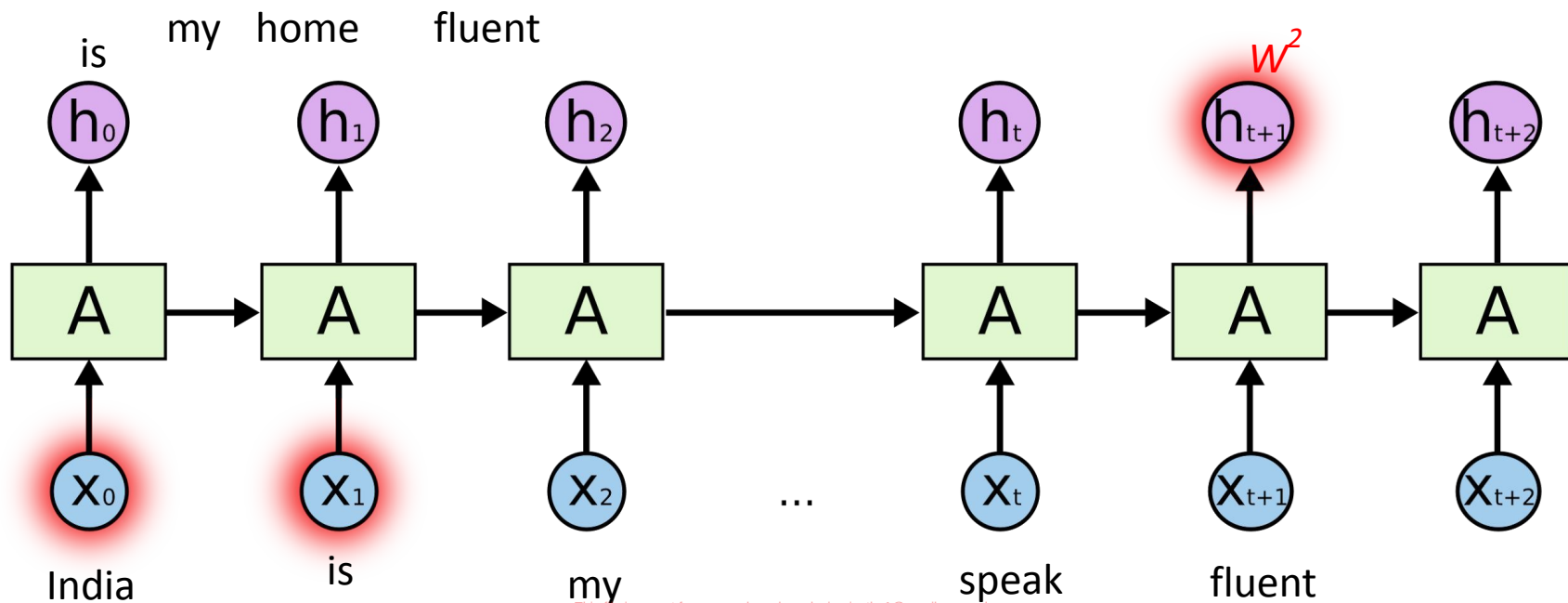
Language modelling: Example - 1

- “the clouds are in the *sky*”



Language modelling: Example - 2

- “India is my home country. I can speak fluent *Hindi*.”

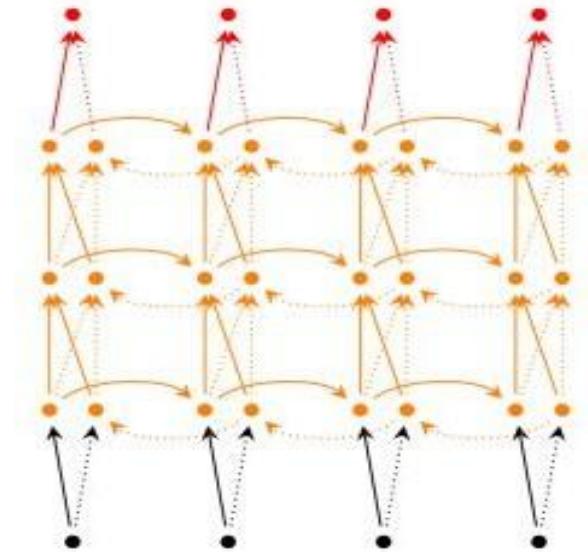
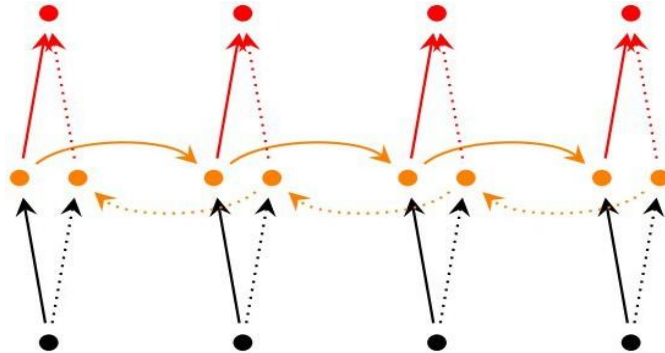


Vanishing/Exploding gradients

- Cue word for the prediction
 - Example 1: *sky* → *clouds* [3 units apart]
 - Example 2: *hindi* → *India* [9 units apart]
- As the sequence length increases, it becomes hard for RNNs to learn “long-term dependencies.”
 - **Vanishing gradients:** If weights are small, gradient shrinks exponentially. Network stops learning.
 - **Exploding gradients:** If weights are large, gradient grows exponentially. Weights fluctuate and become unstable.

RNN extensions

- Bi-directional RNN
- Deep (Bi-directional) RNN



Key Properties of Basic RNN

Maintains a hidden state

Captures short-term dependencies

Uses same weights across time

Works on variable-length input

Suitable for:

time series

speech

text

Extra : Complete BPTT Algorithm



1. Forward Pass:

For $t = 1$ to T :

$$h_t = \tanh(W_{xh} * x_t + W_{hh} * h_{t-1} + b_h)$$

$$y_t = W_{hy} * h_t + b_y$$

$$L_t = \text{Loss}(y_t, \text{target}_t)$$

2. Backward Pass:

Initialize: $\delta_h^{T+1} = 0$

For $t = T$ down to 1:

$$\delta_y^t = \partial L_t / \partial y_t$$

$$\delta_h^t = W_{hy}^T * \delta_y^t + W_{hh}^T * \delta_h^{t+1} \odot (1 - h_t^2)$$

Extra

3. Accumulate Gradients:

$$\partial L / \partial W_{xh} = \Sigma(\delta_h^t * x_t^T)$$

$$\partial L / \partial W_{hh} = \Sigma(\delta_h^t * h_{t-1}^T)$$

$$\partial L / \partial W_{hy} = \Sigma(\delta_y^t * h_t^T)$$

4. Update Weights:

$$W_{xh} \leftarrow W_{xh} - \alpha * \partial L / \partial W_{xh}$$

$$W_{hh} \leftarrow W_{hh} - \alpha * \partial L / \partial W_{hh}$$

$$W_{hy} \leftarrow W_{hy} - \alpha * \partial L / \partial W_{hy}$$

Thank you